

ARTIFICIAL INTELLIGENCE · ADVANCED

Forward Deployed AI Engineering

Move from AI experiments to dependable systems.



Full-Stack / Backend Developer -> GenAI Developer -> Production AI Engineer -> AI FDE.

🕒 8-12 weeks

📁 6 modules

📖 24 lessons

[Explore the course →](#)

Concepts, worked examples, assignments and a coherent course project.

WHO IT IS FOR · WHAT YOU WILL LEARN

Your next capability.

A technology-agnostic, practice-first progression for engineers who build AI applications, productionize them, integrate enterprise systems and own measurable outcomes.



Build LLM, RAG, agent, voice and multimodal applications



Operate AI with evaluation, guardrails, security and observability



Integrate identity, APIs, databases and enterprise systems



Own discovery, adoption and measurable business outcomes



Audience

- Python developers
- C#/.NET developers
- Java/Spring developers
- Node.js/TypeScript developers
- Tech leads and architects



Prerequisites

- Backend programming experience
- Working knowledge of APIs, JSON, Git and databases
- Python is helpful but not mandatory

FROM CONCEPT TO EVIDENCE



One curriculum, three connected views

This guide summarizes the same modules and lessons shown on the course overview and in the learning workspace. The next pages explain the scope and practical evidence for every module.

DETAILED MODULE CURRICULUM · 1-3

Build the foundations.

The module names, lesson sequence and evidence below match the course learning workspace.

1 Engineering Readiness Bridge

Prepare a dependable service boundary before adding a model.

1.1 Backend/API readiness

An API contract specifies inputs, outputs, errors and permissions.

1.2 Data and database readiness

Model transactional records separately from retrieved documents and generated text.

1.3 Testing and Git

Use version control to make changes reviewable and tests to preserve contracts.

1.4 Cloud foundations

A deployed service needs configuration, secrets, logs, health checks and a recovery plan.

You will produce: Service contract, fixture and boundary tests.

2 AI Engineering

Build a grounded answer pipeline for approved knowledge sources.

2.1 LLM foundations

An LLM predicts output from context; fluent language does not prove factual accuracy.

2.2 Prompt and context engineering

Separate task instructions from retrieved evidence and user input.

2.3 Structured output and tools

Structured output makes parsing predictable, but schema validity does not establish semantic correctness.

2.4 Embeddings, vector databases and RAG

Embeddings support similarity search over representations of content.

You will produce: Grounded response set and retrieval evaluation.

3 Agents, Voice and Multimodal AI

Choose bounded orchestration for tasks that combine tools and modalities.

3.1 Agents, tasks and memory

An agent selects steps within an allowed action space.

3.2 MCP, skills and multi-agent patterns

Tool protocols and reusable skills can standardize integration, but they do not grant trust.

3.3 STT, TTS and realtime agents

Voice adds turn-taking, interruption and latency concerns to a text workflow.

3.4 Vision and document AI

Document and image workflows need provenance and uncertainty handling.

You will produce: Orchestration diagram and multimodal failure cases.

DETAILED MODULE CURRICULUM · 4-6

Apply, integrate and demonstrate.

The module names, lesson sequence and evidence below match the course learning workspace.

4 Production AI Engineering

Measure and operate an AI service under realistic failure conditions.

4.1 Evaluations and guardrails

An evaluation set needs representative tasks, expected evidence and clear scoring criteria.

4.2 Docker, CI/CD and cloud

Package a reproducible service and gate releases with tests.

4.3 LLMops and observability

Observe latency, retrieval outcomes, model usage and tool decisions with request identifiers.

4.4 Security, scale and cost

Bound concurrency, token usage and retries.

You will produce: Evaluation report, operational dashboard plan and runbook.

5 Enterprise Integration

Connect enterprise systems without bypassing identity or audit boundaries.

5.1 SSO, OAuth/OIDC and RBAC

Authentication identifies the caller; authorization determines allowed actions.

5.2 Enterprise APIs and databases

An adapter should expose a small, validated contract over the enterprise API or database.

5.3 SharePoint, SAP and CRM integration

SharePoint, SAP and CRM are examples of enterprise information sources, not separate academy consulting offerings.

5.4 Service platforms and legacy systems

Legacy and service-management systems may lack modern event guarantees.

You will produce: Adapter contract, access matrix and audit trace.

6 Solution Architecture and Forward Deployment

Select a solution based on the customer's workflow and evidence needs.

6.1 RAG vs fine-tuning vs SLM

Use retrieval when answers need current external evidence; consider adaptation when a measured behavior needs improvement.

6.2 Agent vs deterministic workflow

A deterministic workflow is preferable when steps and decisions are known.

6.3 Customer discovery and value case

Observe the current workflow, identify users and establish a baseline before estimating value.

6.4 Forward-deployment capstone

The capstone combines discovery, architecture, an adapter, grounded behavior, evaluation and an operations plan.

You will produce: Architecture decision record and forward-deployment portfolio.

PROJECTS AND WORKING METHODS

Enterprise knowledge assistant

Frame a real information need, connect permitted sources, build a grounded assistant and present an evaluation and human-review plan.



Task and integration map



Grounded response examples



Evaluation and operational runbook



Tools and methods

LLM APIs

Vector search and RAG

Agents, MCP and skills

Docker and cloud

Evaluation and observability

Tools are learning choices, not partner endorsements.

Inside module 1: a worked example

```
POST /assistant/query
```

```
Input: {question, conversationId}
```

```
Identity: authenticated server context
```

```
Output: {status, answer, sources, requestId}
```

```
Failures: invalid request; access denied; source unavailable
```

```
Never accept a client-supplied role as authorization.
```

The contract lets the UI handle success and failure without guessing from a generated paragraph.

How your work is reviewed

Show the artifact, explain your decisions, test the stated assumptions and identify limitations. Module assignments build toward the course project rather than replacing it with a single example.

DELIVERY, COMPLETION AND NEXT STEPS

Know what you are signing up for.

**Delivery**

Hybrid learning · 8-12 weeks

Confirm current schedule, cohort arrangements and enrollment terms with the academy.

**Completion requirements**

Complete the required coursework and module assignments, and pass the course assessment.

A course completion certificate is available after those requirements are met.

What the learning workspace includes**Follow the modules**

Expand the curriculum and open the exact lesson you need.

**Apply the concepts**

Read explanations, inspect examples and complete module practice.

**Keep your evidence**

Use resources, notes, discussion and assignments to support your learning.

Learning commitments, not outcome guarantees

Completion is not a placement, employment or performance guarantee. The course project is educational evidence; any production use requires appropriate review and validation.

Start with the curriculum. Continue with the first lesson.

Review your starting point, download this guide and explore the connected course experience.

[Go to the first lesson](#) →

Client-review edition. The static learning workspace saves demonstration progress locally; production enrollment, assessment, communications and certificate issuance are not connected.